AI-Driven Quality Assistance Application for STLC Automation

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Sciences and Business Analytics

BY

Monika Tyagi



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

November, 2026

Declaration of Originality

I, **Monika Tyagi**, with SR No. **13-19-02-19-52-24-1-24515** hereby declare that the material presented in the project report titled

AI-Driven Quality Assistance Application for STLC Automation

represents original work carried out by me at **Alight Services India Pvt Ltd** as part of the project credit requirements of the **Master of Technology (Online)** degree in **Data Science and Business Analytics** at the Indian Institute of Science, between **August, 2024** to **November, 2026**.
With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: Student Signature

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge, and we have carried out due diligence to ensure the originality of the report.

Internal Guide Name: *Kapil Madaan* Internal Guide Signature
Organisation: *Alight Services India Pvt Ltd*

Faculty Mentor Name: *Punit Rathore* Faculty Mentor Signature
Department: *Cyber Physical Systems*

© Monika Tyagi

November, 2026

All rights reserved

Acknowledgements

I thank Mr. Kapil Madaan, my internal project guide at Alight, for the discussions that shaped the problem and for pushing me to keep the prototype close to how testers actually work. I also thank Prof. Punit Rathore, my faculty mentor at IISc, for the reviews on scope and for insisting that the report separate what is implemented from what is only planned.

I am grateful to the faculty and staff of the Indian Institute of Science. Completing the M.Tech. (Online) programme while working would not have been possible without their support, and the comments during report review helped me tighten the limits of this work.

Abstract

In most quality teams I have seen, the work never stays in one system. Requirements sit in Jira. The related change is in GitHub. The last test run is in a log, a pipeline screen, or a spreadsheet someone exported last week. When a tester moves between these, the same facts have to be typed again: what the software is supposed to do, what changed, and what still needs checking. That repetition is a large part of Software Testing Life Cycle (STLC) work, and it is still done by hand.

I took that as the starting point for this project. The question I had was simple enough: can an AI assistant keep one testing task together as it moves from stories and diffs to failures and release notes, while the engineer still makes the call? I implemented a prototype with a React and TypeScript UI, a FastAPI service on PostgreSQL, and a Google Agent Development Kit (ADK) process. The user pastes or fetches context. A root agent then calls one or more of nine specialists. Those specialists write test cases and data, look at a diff, draft a defect, or comment on release risk. Jira Cloud and GitHub can be wired in if credentials are present. Microsoft Entra login and a few model backends are optional.

The rest of the report is the design I actually built, the papers and standards I used, and a plan for measuring the thing properly. I want to be clear about what the software does today. It drafts. It does not run the team's suite. If it suggests a script fix, that fix is not applied to the repo. I could not include a side-by-side benchmark or a timed study in this submission. So I am not claiming that the assistant saves effort or improves traceability. That still has to be checked with people reviewing the output.

Contents

Acknowledgements i

Abstract ii

Contents iii

List of Figures vi

List of Tables vii

List of Algorithms viii

Acronyms, Abbreviations and Symbols x

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement and Motivation . . . 2

1.3 Research Questions . . . 2

1.4 Objectives . . . 2

1.5 Scope and Contributions . . . 3

1.6 Report Organization . . . 3

2 Literature Review 4

2.1 Software Testing Life Cycle . . . 4

2.2 LLMs in Software Quality Engineering . . . 4

2.3 Agent-Orchestrated Workflows . . . 5
2.4 Grounding, Traceability, and Human Review . . . 5
2.5 Evaluation of AI-Assisted Testing . . . 6
2.6 Research Gap and Project Position . . . 6

3 Methodology and Implementation 7
3.1 Development Method . . . 7
3.2 Requirements and Actors . . . 7
3.3 System Architecture . . . 8
3.3.1 Frontend . . . 9
3.3.2 Backend and Persistence . . . 9
3.3.3 Agent Orchestration . . . 10
3.3.4 External Integrations and Model Providers . . . 12
3.4 Security and Data Handling . . . 12
3.5 Deployment and Operations . . . 13
3.6 Quality Attributes . . . 13

4 Results and Discussions 15
4.1 Prototype Implementation Status . . . 15
4.2 Existing Automated Test Assets . . . 15
4.3 Demonstration Scenarios . . . 15
4.3.1 Interface Walkthrough and Illustrative Outputs . . . 17
4.4 Evaluation Plan . . . 21
4.4.1 Dataset and Protocol . . . 21
4.4.2 Measures . . . 22
4.4.3 Analysis and Validity . . . 22
4.5 Results and Discussion . . . 23
4.6 Limitations . . . 23

5 Conclusions **25**

5.1 Future Work 25

Appendix **27**

5.A Representative Request and Response Flow 27

5.A Reproducibility Checklist for a Future Evaluation 27

5.A Project Materials 28

References **29**

List of Figures

3.1 Logical application architecture. Optional external integrations and model providers are enabled by configuration. 8

3.2 Proposed AWS deployment architecture for the Quality Assistance application. This target design is not currently deployed by the repository. 9

3.3 Illustrative agentic workflow showing orchestration among the root agent and specialist agents. 11

3.4 Illustrative request sequence through the application services for the deployment configuration shown. 12

4.1 Workspace dashboard used to start a quality-assistance request. 17

4.2 Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials. 18

4.3 Example request input containing quality context and a user story before agent processing. 18

4.4 Example generated response with prioritized test-case recommendations. 19

4.5 Example suggested test data presented as part of the generated response. 19

4.6 Example automation notes and test-script recommendations. 20

4.7 Example release-risk and readiness analysis based on the submitted context. . . 21

List of Tables

3.1 Selected functional requirements and corresponding implemented behavior. . . . 8
3.2 Registered specialist agents and the scope described by their instructions. 10

4.1 Implementation status as of November 2026, distinguishing current behavior from planned work. 16
4.2 Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed. 22

List of Algorithms

1 Assistance request processing 11

Acronyms, Abbreviations and Symbols

These short forms are used in more than one chapter.

Acronyms

ADK Agent Development Kit

API Application Programming Interface

CI/CD Continuous Integration and Continuous Delivery

LLM Large Language Model

M.Tech. Master of Technology

MCP Model Context Protocol

QA Quality Assurance

RQ Research Question

STLC Software Testing Life Cycle

UI User Interface

Abbreviations

E2E End-to-end

JWT JSON Web Token

ACRONYMS, ABBREVIATIONS AND SYMBOLS

RAG Retrieval-Augmented Generation

REST Representational State Transfer

Chapter 1

Introduction

1.1 Background

Running tests is only one part of software quality. In the Software Testing Life Cycle (STLC) the team first reads the requirement, decides the scope, designs cases and data, executes, logs defects, writes a summary, and then, often under time pressure, says whether a release can go. ISO/IEC/IEEE 29119 and the ISTQB Foundation syllabus both treat this as planned work that should stay traceable to risk [1, 2].

The part that slowed people down, in my experience, was not any single step. It was the spread across tools. A story arrives as a ticket. A change arrives as a diff. A failure arrives as a log. Somebody still has to turn the story into tests, decide what the diff should regress, and turn the run into something a release manager can read. On one ticket this is ordinary. Across a sprint it is the same interpretation done again and again, and the wording drifts.

Large language models (LLMs) handle the language part of that reasonably well. They can summarise a story, suggest boundary cases, or drop a messy log into a defect template. They are weaker when no one is watching. I have seen models skip a rule that was in the second paragraph, add a field that was never there, and still sound certain. For a quality team that is a problem. An assistant is useful only if it takes the context the user gives it, stays on the task, shows the draft, and waits for a person to accept or throw it away.

1.2 Problem Statement and Motivation

I started from a gap I kept hitting at work. Teams already have bits of automation: a generator for cases, a formatter for reports, maybe a linter for scripts. What they still do manually is join the requirement, the diff, the result, and the release note. Analysis, triage, and readiness usually sit outside the pipeline. Context falls between tools. A scenario is missed. Risk shows up in a late review instead of in the original plan.

A single prompt in a chat window did not feel like enough, because the jobs are different. So I built an assistant that can call specialists. The user can send requirements and stories, a source-code diff, or a test-run log. A root agent picks who should answer and returns a draft. The engineer still reads the cases, runs tests in the right environment, and decides on the release.

1.3 Research Questions

I kept the three questions from the project proposal and the mid-term presentation:

1. **RQ1:** How can a multi-agent AI system coordinate several STLC activities through one quality-assistance interface?

2. **RQ2:** What validation, provenance, and human-review controls are needed before AI-generated testing artifacts can be trusted enough to use?

3. **RQ3:** How should the system be evaluated for its effect on requirement-to-test traceability, quality-engineering effort, coverage, and defect analysis?

1.4 Objectives

The practical goals were:

- put up a full-stack application where a user can submit quality context and read an AI-assisted reply;

- put a central agent in place, with specialists for the STLC jobs I needed;

- store requests against the signed-in user and keep the app behind login;
- pull context from Jira and GitHub when those are configured;
- leave the model provider and deployment settings in the environment, so they can change without a rewrite; and
- write an evaluation plan that can be repeated, with human review, and not report benefits I have not measured.

1.5 Scope and Contributions

This report is about the code in the repository as of November 2026. What I am submitting is a working prototype. I am not offering a new theory of agents. The browser UI, the API, and the agent service run together. Users can paste requirements and stories, attach a diff or a log, continue in a later turn, and export the reply. If Jira Cloud and GitHub are set up, issue text and diffs can be fetched instead of typed. The root agent then works with nine STLC specialists.

There are limits I do not want to hide. The prototype will not run someone else's suite. It will not rewrite scripts in a git repo. It cannot prove that the generated cases are complete, and it cannot approve a release. Execution, self-healing, and readiness answers are suggestions from the text that was submitted. I also did not implement retrieval over a test repository, CI/CD execution, strict output schemas, or a comparative study. Those are listed later as future work.

1.6 Report Organization

Chapter 2 is the reading I used: STLC, LLMs in testing, agent setups, and the validation issues. Chapter 3 is how I built the prototype, including architecture and security. Chapter 4 is what I can actually show from the repo, plus the evaluation I still need for the research questions. Chapter 5 is the wrap-up and the list of next steps.

Chapter 2

Literature Review

2.1 Software Testing Life Cycle

Testing is supposed to be planned. The point is to learn something about the product and leave a record, not just a pass count on the last day of the sprint. ISO/IEC/IEEE 29119 describes the concepts, processes, and documents. ISTQB Foundation covers the same ground in the language testers use: planning, analysis, design, implementation, execution, completion [1, 2]. In both places the same idea comes back. A test condition should point to a requirement or a risk, and a conclusion should rest on something that was written down.

The STLC on a real project is not as tidy as the diagram. A failed run can show that the requirement was vague. A defect can force extra regression. A late commit can make the original scope wrong. If an assistant is going to help in more than one of these phases, it has to carry context. If every answer is generated as if the previous one never happened, we are back to the same fragmentation the tools already have.

2.2 LLMs in Software Quality Engineering

LLMs can read a story and write scenarios. They can suggest data, classify a failure, or summarise a run. That is why people reach for them: the jobs are repetitive and mostly text. Reliability is a different matter. A model may ignore a constraint, invent a step, or write as if it is sure when the input is thin.

Amershi et al. make a similar point for machine learning in software teams: the workflow has to stay with people, ownership has to be clear, data quality has to be watched, and the system has to be monitored after it ships [3]. I treated that as something I had to design for. In this assistant the model output is a draft. A tester should be able to check the assumptions, edit the cases, and drop a recommendation that is not supported, before it goes into a plan or a release discussion.

2.3 Agent-Orchestrated Workflows

The usual agent pattern is a set of specialists and a coordinator that decides who to call. ReAct is one way of mixing reasoning with actions. Broader surveys talk about planning, tools, memory, and how agents are coordinated [4, 5]. Splitting the work made prompts easier for me to maintain. It did not make the answers correct. More calls also mean more latency and more cost, and the coordinator can pick the wrong specialist or call too many of them.

My prototype has a root agent and a registry of STLC specialists. Routing depends on the request and the files or text attached to it. When I read the code for this report, that routing was still mostly the instructions plus whatever the model generated. The split helped me implement the thing. It is not a workflow engine, and nobody has verified it formally. The next hardening I would do is schemas between stages, some validation, and a place where a person has to approve.

2.4 Grounding, Traceability, and Human Review

Retrieval-Augmented Generation (RAG) is the usual answer when a model needs facts it should not invent. Lewis et al. retrieve documents and then generate from them [6]. In testing that would mean using approved requirements, policies, API specs, or old defects, instead of only the pasted prompt. I already marked this as later work in the proposal and the mid-term. The repo I inspected for this report has no persistent index over test assets. Right now the prototype uses what the user attached, and whatever is still sitting in the in-memory session.

Even with retrieval, an evaluation has to tell two things apart: whether a case can be traced to a requirement, and whether the case is actually right. People still have to catch gaps and

unsupported claims. If the output could affect a release, the evidence should stay on screen, uncertainty should be visible, and someone should have to accept it.

2.5 Evaluation of AI-Assisted Testing

If I were running the study now, I would want a baseline, inputs that look like real tickets, a frozen model and prompt, and scores that can be repeated. The measures I care about are requirement-to-test links, scenario coverage, human-rated accuracy, time on the same tasks, defect classification where labels exist, and end-to-end latency. They are not interchangeable. A longer list of cases is not better coverage. A faster first draft is not less work if review and correction take the time back.

NIST's AI Risk Management Framework organises this around governance, mapping, measurement, and management [7]. For me that meant: record the model and config versions, write down how a reviewer should score an artifact, keep suggestions separate from verified results, and treat privacy and operational risk as part of the evaluation, not something to add later.

2.6 Research Gap and Project Position

The proposal and mid-term put the gap in one sentence: we can automate single QA tasks, but the work that joins requirements, design, diffs, failures, and release evidence is still loosely coupled. The prototype is one application with a coordinating agent that can call those task-specific pieces. I am not saying agent orchestration is a new research idea. What I built is the software. Whether the coordination is useful, whether the output is dependable, and whether the workflow beats a baseline that people can describe, is what a later evaluation has to show.

Chapter 3

Methodology and Implementation

3.1 Development Method

I did not start from a frozen specification. First I tried to write down the quality-workflow problem in a way that would survive review. Then I sketched services. After that I implemented the user path, then the agent path. Only later did I sit down and ask what evidence I would actually need to evaluate either. The proposal already said evaluation should be phased: public and open-source material first, and only then, if approved, anonymized work artifacts. The mid-term slides are an earlier snapshot of the prototype and of that plan. This chapter is the implementation. The comparison study is still ahead.

When I wrote the "as built" sections I went through the frontend, backend, agent service, tests, and deployment notes. If I could not find a path in the code, I did not report it as done. The proposal and presentation say what I intended. They are not a substitute for experiments I have not run.

3.2 Requirements and Actors

The person I had in mind is a quality engineer, a developer, or someone on a release call who wants structured testing help, not another open-ended chat. After login they provide requirements or stories, maybe a diff or a test-run log, and then they read the reply. A follow-up can reuse a session id. An administrator picks the model and turns Jira or GitHub on

through environment settings.

Requirement	Prototype behavior
Context submission	Accepts requirements, user stories, code diffs, and optional session context through the authenticated assistance API.
Quality assistance	Returns a generated response that can include prioritized cases, test data, automation notes, and release risks.
Specialized analysis	Routes suitable requests to registered STLC agents for test design, execution-log analysis, change impact, defects, summaries, and readiness.
User separation	Associates assistance requests with the authenticated user and persists request inputs and responses in PostgreSQL.
Engineering context	Offers Jira Cloud story retrieval and GitHub pull-request or branch-diff retrieval when configured.
Operational configuration	Supports provider/model selection, containerized local startup, health checks, and production configuration checks.

Table 3.1: Selected functional requirements and corresponding implemented behavior.

3.3 System Architecture

There are four pieces: the browser UI, the API, the database, and the agent process. The React and TypeScript frontend calls the FastAPI backend. After the user is authenticated, the backend stores the request, builds the agent message, and later stores the reply. A separate service runs the root agent and the specialists through Google ADK. PostgreSQL holds users, requests, and the integration records.

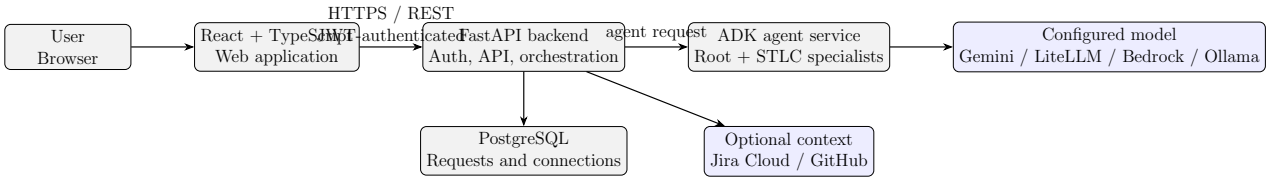


Figure 3.1: Logical application architecture. Optional external integrations and model providers are enabled by configuration.

Figure 3.1 matches what is in the repo. The AWS picture after it is a target I wrote up, not something that is running. The repo has notes for that layout. It does not create the accounts or the cluster.

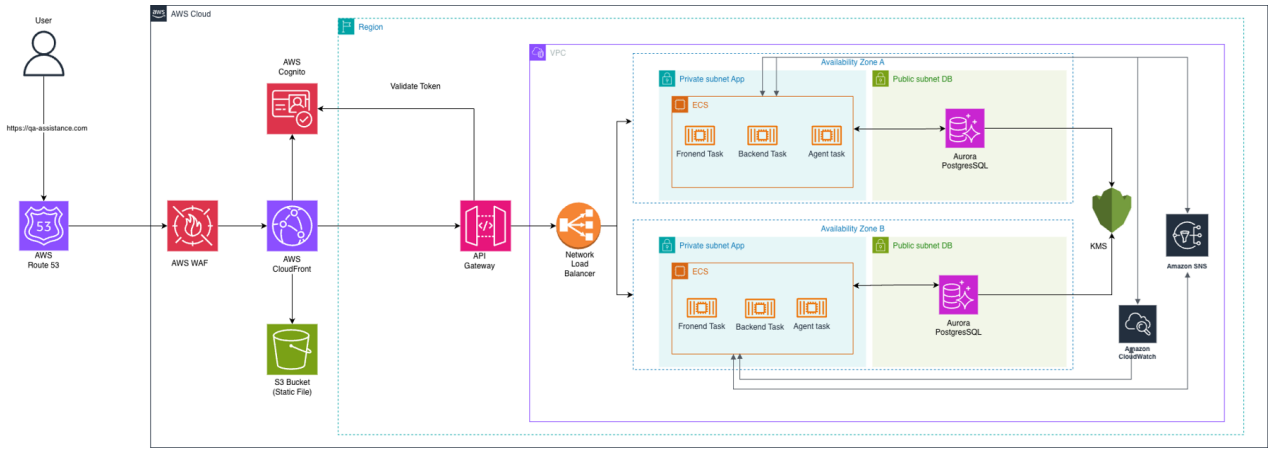


Figure 3.2: Proposed AWS deployment architecture for the Quality Assistance application. This target design is not currently deployed by the repository.

3.3.1 Frontend

The UI has login, a dashboard for requests, a settings page, and a short user guide. On the dashboard the user can type requirements and stories, paste a change, or start from the samples I added: smoke planning, an end-to-end example, change impact, execution and reporting, automation, and a follow-up. If Jira is connected, assigned issues can fill the story field. If GitHub is connected, a pull-request or compare diff can be fetched. The reply is rendered in the browser. It can also be downloaded as Markdown or HTML.

3.3.2 Backend and Persistence

The backend is FastAPI, with asynchronous SQLAlchemy on PostgreSQL [8]. `/api/assist` needs a logged-in user. The request is stored with owner, source text, session id, status, timestamps, and later the response. Then the backend builds a structured message and calls the agent service. Both success and failure are written back. Errors go to the frontend on the normal error path.

Users, assistance requests, Jira connections, and GitHub repo connections are separate tables. That is enough to open an old submission. It is not enough if you expect the agent conversation to survive a restart. The ADK runner I used keeps session state in memory. I found that out the hard way: after a restart, a follow-up could lose context that was still on

the screen.

3.3.3 Agent Orchestration

The root agent is `quality_assistance_agent`. I built it with Google ADK, the configured model, logging callbacks, and the tools available at start-up [9]. On the supported cloud providers the root agent can pass work to the registered sub-agents. On local Ollama I used one combined instruction instead, because that path does not transfer the specialists. The backend prompt asks for prioritized cases, suggested data, automation notes, and release risks, using the requirements, stories, and diffs that came in.

ID	Specialist	Implemented responsibility
S01	Test case generation	Drafts structured, prioritized cases from requirements and stories, including positive, negative, and boundary scenarios.
S02	Test data generation	Suggests representative and edge-case test data.
S03	Test script automation	Suggests automation approaches and repeatable script outlines.
S04	Test execution	Interprets supplied or simulated test-run results and highlights failures; it does not itself launch the user's test suite.
S05	Self-healing test scripts	Recommends maintenance strategies when selectors or interfaces change; it does not directly edit repository files.
S06	Change impact analysis	Relates supplied code diffs to potentially affected areas and regression scope.
S07	Bug report generation	Structures failure information into an actionable defect report.
S08	Test summary reporting	Summarizes test outcomes for stakeholders.
S09	Release readiness evaluation	Weighs supplied test and risk signals and provides a reasoned recommendation.

Table 3.2: Registered specialist agents and the scope described by their instructions.

Requirement and story text go to the test-case and test-data agents. Diffs go to change impact. Execution logs go to execution, defect, and summary. Release questions go to readiness. The root agent also has a small requirement-summarization tool. All of this is prompt routing. There is no second engine underneath it that would force the same path every time.

3. METHODOLOGY AND IMPLEMENTATION

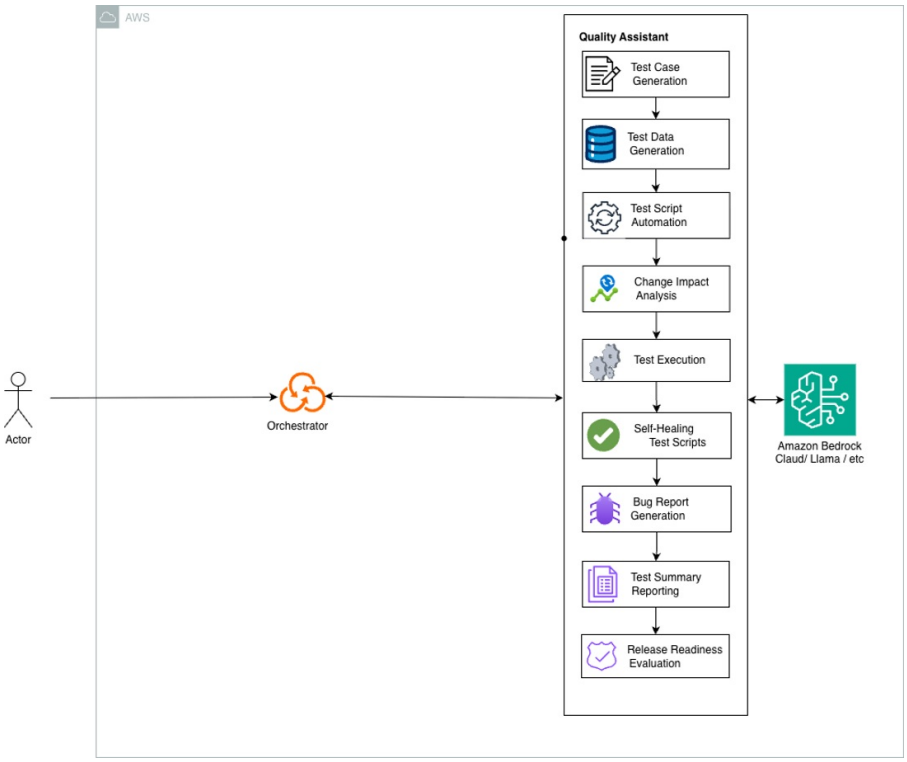


Figure 3.3: Illustrative agentic workflow showing orchestration among the root agent and specialist agents.

Algorithm 1 Assistance request processing

Require: Authenticated user u, requirements r, optional stories s, diff d, session q

Validate the request and associate it with u
Persist request inputs with status `processing`
Construct a message from r, s, and d
Resolve or create an agent session for u
Invoke the root agent with the message and session
Route to one or more specialists according to the request context
Collect the generated response from the agent events
if agent call succeeds **then**
    Persist session identifier, response, and status `completed`
else
    Persist failure status and return an explicit service error
end if
Return the request identifier, session, response, and status

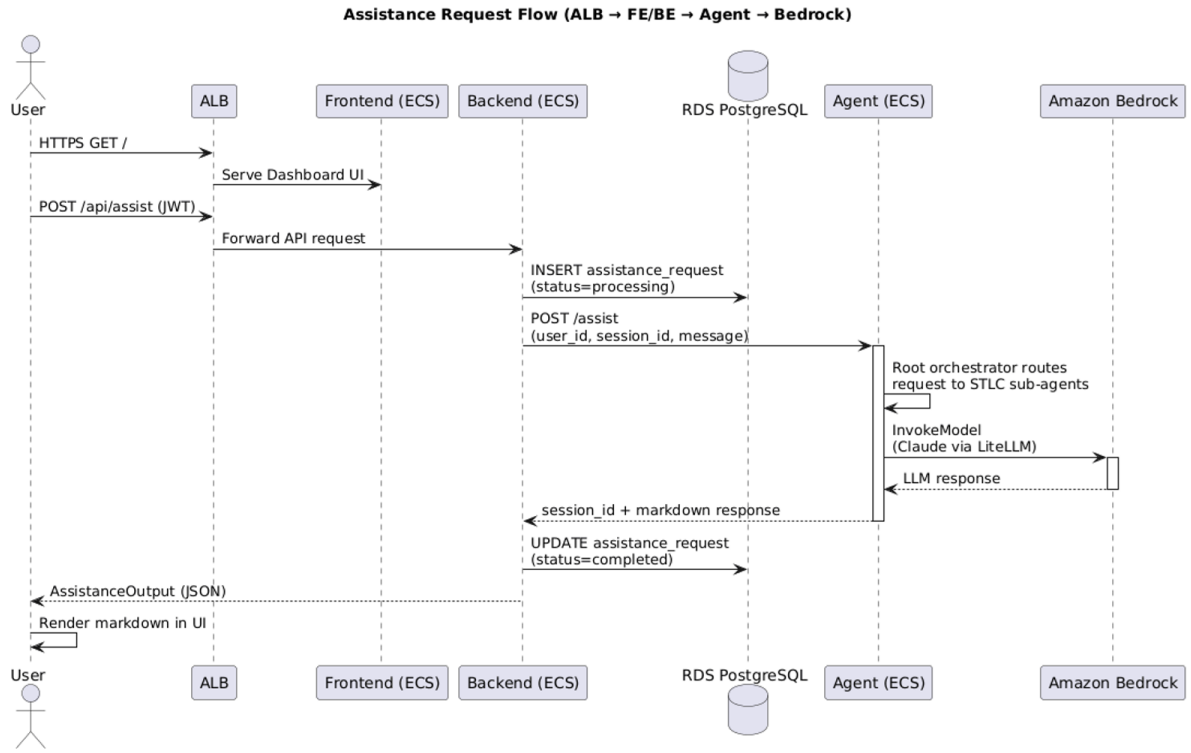


Figure 3.4: Illustrative request sequence through the application services for the deployment configuration shown.

3.3.4 External Integrations and Model Providers

Jira Cloud is OAuth 2.0, a short-lived state record, and Jira APIs to list assigned issues and fetch detail. GitHub settings store a repo and an encrypted personal access token. From there the backend can pull a pull-request diff or compare two references. Microsoft Entra SSO is optional. The agent side can use Gemini, LiteLLM, Amazon Bedrock, or Ollama if the credentials and models are there. I put those options in so the same code can run in more than one place. I did not benchmark every provider on the same tasks.

3.4 Security and Data Handling

Assistance records are scoped to the signed-in user. JWT tokens are issued after register or login. Passwords go through the project's password manager. Jira and GitHub credentials are stored encrypted. Jira OAuth state is a digest and it expires. In production the settings check

expects real secrets, an encryption key, explicit HTTPS origins, database TLS, and non-default database credentials. The ECS note describes private app and database services, IAM roles, and managed secrets.

Requirements, stories, diffs, and test results are often confidential. The application stores them with the request and sends them to the configured model provider. Anyone who deploys this has to pick a provider and a retention policy the organisation will accept, limit who can see stored data, rotate credentials, and keep secrets out of the prompt. The checks I added reduce accidental leaks. They are not a privacy assessment, a threat model, or an internal security sign-off.

3.5 Deployment and Operations

For local use there is a Docker Compose file with PostgreSQL, frontend, backend, and agent. There are also scripts if you want the UI and Python services on the host, with the database local or in a container. The AWS ECS Fargate note is a pilot layout: separate containers, RDS PostgreSQL, an Application Load Balancer, and a chosen model. It is a checklist. It is not Terraform or CloudFormation, and I am not claiming that an AWS production stack has been brought up.

3.6 Quality Attributes

- **Modularity:** frontend, API, database, and agent run as separate processes. Specialists are registered on their own.
- **Configurability:** model, endpoints, integrations, and production flags come from the environment.
- **Traceability:** each user's inputs and replies are stored. I still do not have a formal requirement-to-case trail, and agent memory is not durable.
- **Availability:** health and readiness endpoints are there, and the container notes help. How well a real install holds up depends on the topology someone actually chooses.

- **Reliability:** the backend has error paths and settings checks. Generated test artifacts still need a person to read them, and they still need a proper study.

Chapter 4

Results and Discussions

4.1 Prototype Implementation Status

What I have at this point is a running full-stack prototype. It is more than a slide architecture. A user can register, sign in, send context through the authenticated API, and read a reply on the dashboard. Requests are stored. Nine specialists are registered, and the root agent picks among them from its routing instructions. GitHub diff retrieval, Jira Cloud OAuth and issue fetch, and optional Microsoft Entra login are in the same repo.

4.2 Existing Automated Test Assets

I wrote focused tests for Bedrock credentials, backend production settings, Jira Cloud behaviour, and Microsoft Entra SSO. Those files show that a few paths mattered enough to get checks. They do not tell you the suite is green today, and they do not cover the UI, the model, the database, and the agents end to end. If a later version of this report is going to claim test evidence, it should include the command, the environment, the commit, the test count, and the actual result of a run.

4.3 Demonstration Scenarios

The dashboard includes samples I used while building, so I could hit several agent routes without inventing a ticket every time:

Project area	**Status in inspected repository**	**Evidence or qualification**
Problem analysis and architecture	Implemented	Research framing is documented in the proposal and presentation; current services are separated into frontend, backend, and agent.
Core quality-assistance flow	Implemented	Authenticated request persistence, prompt construction, agent call, and response persistence are present.
Specialist STLC capabilities	Implemented	Nine task-focused agents are registered; orchestration is controlled by ADK and agent instructions.
Jira and GitHub context	Implemented, configuration-dependent	OAuth and issue retrieval are available for Jira; GitHub repository settings support diff retrieval.
Authentication and deployment controls	Implemented in part	Password/JWT authentication, optional Entra SSO, encrypted integration credentials, and production settings validation are present.
RAG over organizational test assets	Planned	No persistent requirements/test/defect retrieval index is evidenced in the inspected implementation.
Automated CI test execution and script mutation	Planned / out of scope of current behavior	Execution and healing agents analyze or recommend; they do not execute external suites or commit changes.
Comparative empirical evaluation	Pending	No measured baseline, benchmark results, or controlled organizational pilot results were supplied.

Table 4.1: Implementation status as of November 2026, distinguishing current behavior from planned work.

1. a password-reset requirement, for planning and summarising;
2. an e-commerce checkout with requirements, stories, and a diff;
3. an authentication change with a large diff, for regression impact;
4. a test-run log with pass, fail, and skipped cases, for execution analysis and reporting;
5. a Playwright selector change, for automation and self-healing notes; and
6. a follow-up that reuses the previous session id.

They were useful for demos. They are not a benchmark. They do not measure coverage, effort, or defects found.

4.3.1 Interface Walkthrough and Illustrative Outputs

The screenshots are one ordinary path: open the workspace, check settings, type a requirement and a story, then read the reply. They show the screens and a sample output. I did not run a controlled evaluation on them. The recommendations still need a person.

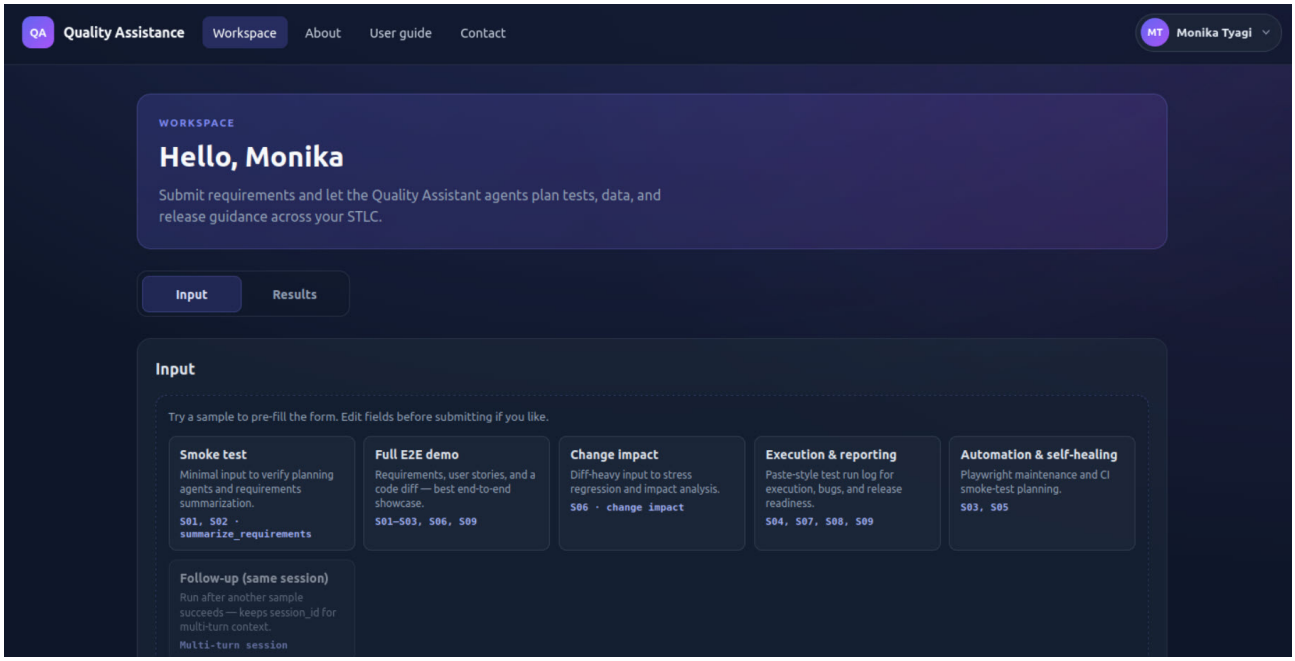


Figure 4.1: Workspace dashboard used to start a quality-assistance request.

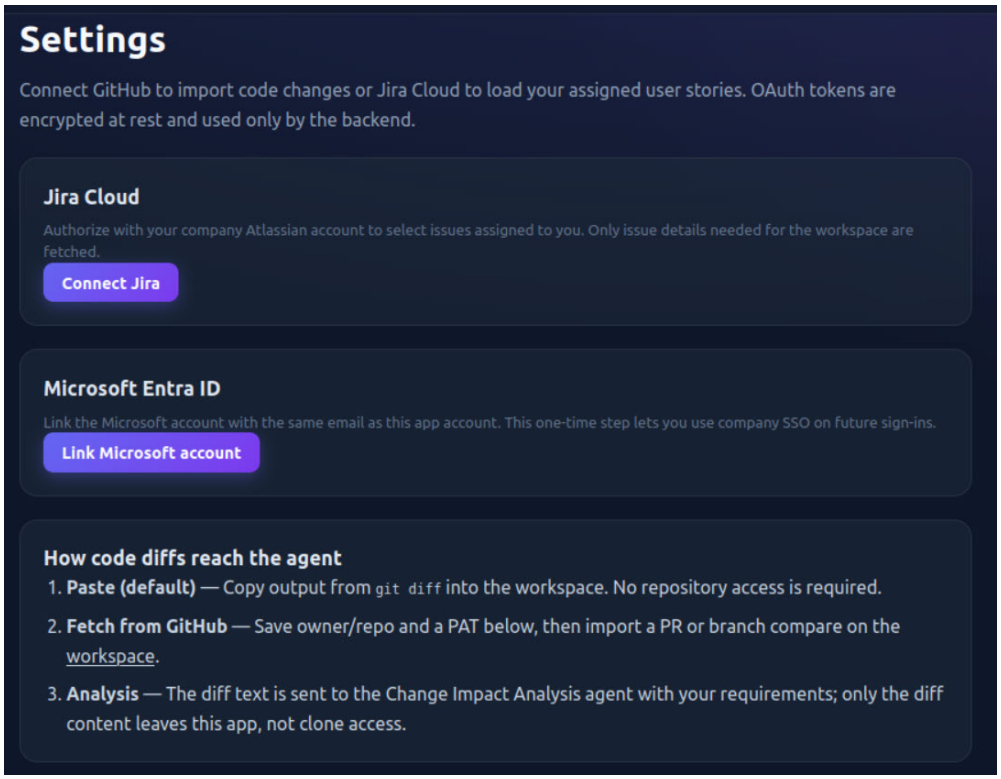


Figure 4.2: Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials.

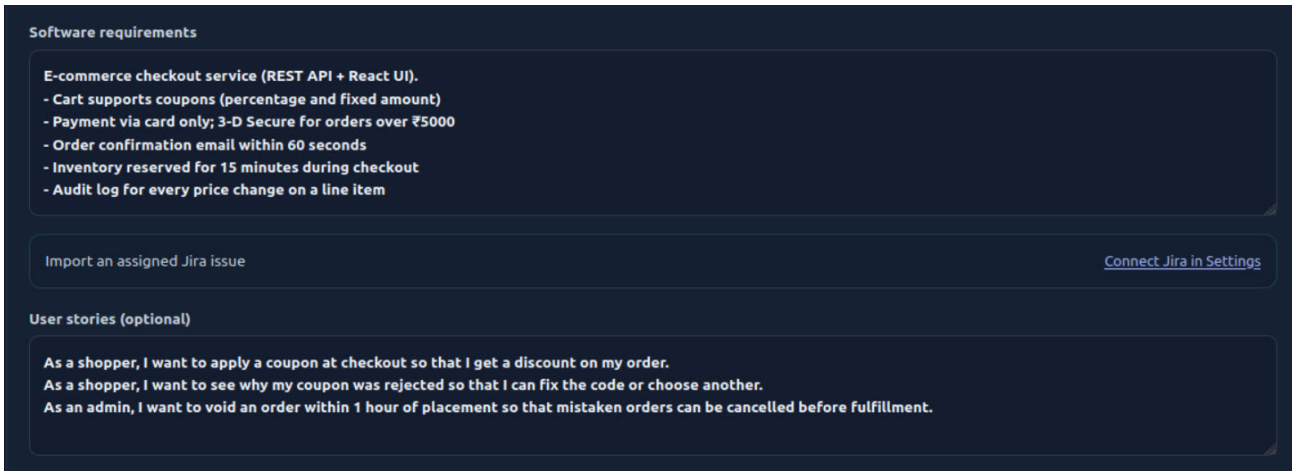


Figure 4.3: Example request input containing quality context and a user story before agent processing.

4. RESULTS AND DISCUSSIONS

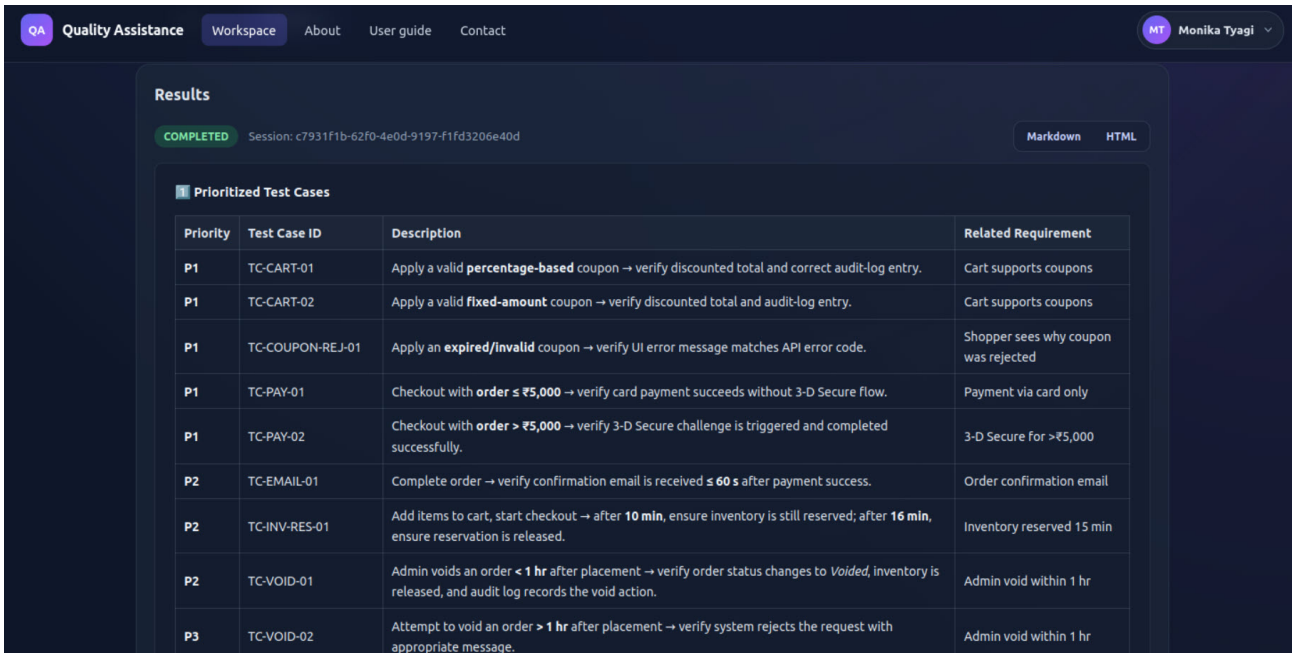


Figure 4.4: Example generated response with prioritized test-case recommendations.



Figure 4.5: Example suggested test data presented as part of the generated response.

4. RESULTS AND DISCUSSIONS

3 **Automation Notes** (for **test_script_automation** agent)

Area	Automation Approach	Script Outline / Tools
API Tests	Use **Postman/Newman** or **REST-Assured** for end-to-end flow (add to cart → apply coupon → checkout).	• Parameterize coupon codes, payment amounts • Validate response codes & JSON schema • Capture `orderId` for downstream checks
UI Tests	Selenium/WebDriver (or Cypress) for React checkout UI.	• Locate coupon input, submit and assert error/discount message • Mock 3-D Secure redirect via test gateway • Verify email-sent toast and order-status page
Email Verification	Use **MailHog** or **SMTP mock server** to capture outbound emails; assert receipt time.	• Start timer before order finalization • Poll mock server for matching `orderId` email • Assert timestamp ≤ 60 s
Inventory Timer	Background job simulation using **time-travel** in test environment (e.g., `clock` mock).	• After checkout, fast-forward 16 min and query inventory status via API • Verify reservation released
Audit Log Validation	Direct DB query or log-collector API (ELK).	• After price change, query `audit_log` table for recent entry • Assert fields: user, oldPrice, newPrice, timestamp
Admin Void	API call with admin token; combine with UI flow for admin portal.	• Submit void request within 1 hr, assert status `Voided` • Attempt after 1 hr, assert error message
CI/CD Integration	Add to **GitHub Actions** pipeline: 1 Unit → 2 API contract → 3 End-to-end UI (headless Chrome) → 4 Post-run report.	Use environment variables for secrets (payment gateway mock keys).
Self-Healing (future)	Tag UI elements with stable `data-testid` attributes; configure selector fallback strategy.	Include in **self_healing_test_scripts** recommendation later.

Figure 4.6: Example automation notes and test-script recommendations.

4️⃣ **Release Risks & Mitigations** (for **release_readiness_evaluation**)

Risk	Impact	Likelihood	Mitigation
Coupon mis-calculation (percentage vs fixed)	Wrong discount → revenue loss, poor UX	Medium	End-to-end price verification in every build; unit tests for discount engine
3-D Secure failure for >₹5,000 orders	Payment decline, cart abandonment	High (depends on external gateway)	Mock gateway in staging; monitor real-time success rate after release
Email latency >60 s	Customer confusion, support tickets	Medium	Deploy email service with SLA; add alert on queue depth; include performance test in CI
Inventory reservation timeout bug	Overselling, fulfillment issues	Low to Medium	Automated timer test; log monitoring for "reservation expired" events
Audit log omission	Compliance breach, debugging difficulty	Low	Enforce log write via middleware; unit test audit interceptor
Admin void window logic error	Orders stuck in pending state, financial reversal problems	Medium	Regression suite covering <1 hr and >1 hr cases; code review of time-window logic
Concurrency coupon application	Race condition causing double discount	Low	Stress test with parallel users; lock cart updates at DB level
React UI regression (broken data-testid)	Automation scripts break, manual testing overhead	Medium	Adopt `data-testid` convention; run UI regression smoke after each UI change

Overall Release Readiness – If all **P1** test cases pass in CI, email latency ≤ 45 s in load test, and audit-log coverage is 100 %, the release can be considered **green**. Any failure in P1 or a missed SLA (email, 3-D Secure) should halt promotion to production.

Figure 4.7: Example release-risk and readiness analysis based on the submitted context.

4.4 Evaluation Plan

When I do the study, I want three conditions on the same tasks: a person doing the work by hand, whatever point automation the team already has, and this assistant. The first round should stay on public requirements and open-source repos or trackers, which is what the project plan already said. If we later use organisational material, it has to be anonymized, and it has to wait for the data-governance approval that organisation needs.

4.4.1 Dataset and Protocol

For each project I would pick a small, closed set of requirements, stories, code changes, and any test-run or defect records that already exist. Expected requirement-to-test links and expert-reviewed reference outputs should be written down before the assistant is run. Model, model version, prompt revision, and app config stay fixed for a comparison. Because the model does

not always give the same answer, I would repeat tasks where I can and keep both the raw output and the review decision. If it is practical, the person scoring the artifacts should not know which workflow produced them.

4.4.2 Measures

Measure	Operational definition	Interpretation
Requirement traceability	Percentage of in-scope requirements linked to at least one reviewer-accepted test case.	Tests whether generated artifacts preserve requirement coverage links.
Scenario coverage	Reviewer-rated coverage of agreed positive, negative, boundary, and risk conditions.	Measures breadth and relevance rather than the raw number of generated cases.
Artifact correctness	Proportion of sampled cases judged correct and actionable by independent reviewers.	Captures factual and procedural quality; report agreement between reviewers.
Effort	Active authoring and review time per requirement or change.	A reduction is meaningful only if review and correction time are included.
Defect analysis quality	Precision, recall, or F-score against adjudicated defect categories when labels exist.	Applies only where a suitable labeled reference set is available.
Response time and cost	End-to-end latency and model usage per completed task.	Quantifies operational overhead and helps assess scalability.
User acceptance	Structured ratings of usefulness, clarity, and trust from representative practitioners.	Complements objective measures but should not replace correctness review.

Table 4.2: Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed.

4.4.3 Analysis and Validity

For numbers I would report sample size, the shape of the data, confidence intervals, and which tests I used. I would also look at error types, not only averages. On the qualitative side I would look for missing cases, claims with no support, unclear requirements, and places where the model and an expert disagree. Model and prompt versions have to be written down, otherwise a rerun later is hard to explain.

The threats I worry about are the usual ones. The dataset may not look like the work

people actually do. Reviewers will disagree. The model will not give the same answer twice. The provider may change the model under us. Examples may leak into prompts. And a long generated list may simply create more review. A small public-data run would be enough to talk about feasibility. It would not be enough to talk about enterprise workflows.

4.5 Results and Discussion

The evidence in this chapter is that the software exists and can be exercised. It takes quality context, stores a request, calls the orchestrated agent, and returns a response. The specialist list and the dashboard samples cover several STLC jobs. The Jira and GitHub code is a starting point for systems a team already has.

I do not have experiment results to put next to that. No benchmark set, no time-saved numbers, no coverage figures, no defect-detection rates, no user study. So I am not claiming the system improves quality or reduces effort. What this stage produced is the application and a way to test those claims later. A generated case can look complete and still be wrong, incomplete, or useless for finding defects.

4.6 Limitations

- If the user gives thin or wrong context, or if the chosen model is weak, the recommendations follow that.
- Routing between agents is still mostly the prompt. I did not add formal schemas or independent quality gates between stages.
- Session state is in memory. Restart the agent process and the conversation is gone.
- The system does not run external suites, edit scripts, or block a release.
- Retrieval over requirements, test assets, and old defects, and a Jenkins or similar CI hook, are still to be done.
- Tests cover a few settings and services. UI, model quality, load, and a full end-to-end path still need work.

- I have planning notes for production, not a signed-off deployment or an organisation-specific privacy review.

Chapter 5

Conclusions

The aim was to keep several STLC jobs in one place. The repo now has a full-stack application that does that, at least as a draft workflow. The React and TypeScript UI, the FastAPI and PostgreSQL backend, and the Google ADK agent service meet at a root agent that can send work to nine specialists. Those specialists write test design and data, comment on a change, draft a defect, and write release-readiness notes. Login, request storage, Jira Cloud and GitHub context, optional Microsoft Entra SSO, and a choice of model providers are also there.

Getting those pieces to talk to each other was the engineering work. It is not the same as showing that quality improved. I have not run a controlled comparison, so I cannot say the assistant improves coverage, cuts the time a tester spends, or finds more defects. The agents write drafts. Someone still has to judge them. Execution analysis does not launch a suite. A self-healing note does not change the code. A readiness paragraph does not approve a release.

5.1 Future Work

What I would do next, in order:

1. structured outputs and checks between specialist stages;
2. a place for a person to approve or edit test cases, and anything that could affect a release;
3. an evaluation of traceability, correctness, review effort, latency, and cost against a baseline written down in advance;

4. trials on public projects and, after approval, on anonymized organisational artifacts;
5. retrieval over approved requirements, test repos, and historical defects;
6. a careful CI/CD hook for test execution, and session history that survives a restart and can be audited; and
7. load, reliability, accessibility, privacy, and security reviews before this is treated as a wider deployment.

I already know an LLM can write something that looks like a test case. The question I still cannot answer is whether this orchestrated assistant produces work that people can trace and use, often enough that the review it creates is still cheaper than doing the task without it. The prototype is the place to run that comparison. Screenshots of a working demo are not the comparison.

Appendix

5.A Representative Request and Response Flow

A request usually starts after login. The user enters some mix of requirements, stories, and a diff. The frontend posts that to the backend assistance route. The backend attaches the current user, saves the request as processing, and builds an agent message. The agent service reuses a session if one exists, otherwise it opens a new one, calls the root agent, and collects text from the events. The backend stores the reply and the final status and returns the request and session ids. For a follow-up, the same session id goes back with the next turn.

5.A Reproducibility Checklist for a Future Evaluation

- Write down the application revision, model provider, model id, prompt revision, and runtime config.
- Keep anonymized inputs and the reference cases used for scoring.
- Use the same tasks and the same review rubric for every baseline and for the assistant.
- Time generation separately from editing and review.
- Track what was accepted, edited, or rejected, and why.
- Report the scores and the failures. Do not drop unsafe or misleading outputs.
- Get approval before using organisational requirements, source code, or defect records.

5.A Project Materials

The proposal has the motivation, the questions, the phased evaluation, and the work plan. The mid-term presentation has the agent framework I intended at that point and the status then. The implementation and deployment notes in the repo are what I used for the as-built architecture and for the feature limits in this report.

References

[1] International Organization for Standardization. *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering—Software testing—Part 1: General concepts*. ISO, 2022. 1, 4

[2] International Software Testing Qualifications Board. Certified tester foundation level syllabus, version 4.0. https://www.istqb.org/certifications/certified-tester-foundation-level, 2023. 1, 4

[3] Saleema Amershi, Andrew Begel, Christian Bird, Robert De Line, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300, 2019. doi: 10.1109/ICSE-SEIP.2019.00042. 5

[4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 5

[5] Lei Wang, Cheng Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wei Zhao, Zhe Wei, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. 5

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. 5

[7] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023. 6

[8] Sebastián Ramírez. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed November 2026. 9

[9] Google. Agent development kit documentation. https://google.github.io/adk-docs/. Accessed November 2026. 10